Session: Introduction to NumPy from Scratch

1. What is an Array?

Before understanding NumPy, we must first understand the concept of an array.

An array is:

- A collection of elements
- Stored in a structured manner
- Typically of the same data type
- Arranged in rows and columns (optional)

In simple terms:

An array stores multiple values under a single variable name.

Example of a 1-Dimensional Structure

A list in Python:

[10, 20, 30, 40]

This looks like an array.

However, a Python list:

- Can store mixed data types
- Is not optimized for numerical computation
- Is slower for mathematical operations

---

2. Why Do We Need NumPy?

When working with:

- Large numerical datasets
- Scientific computation
- Matrix operations
- Machine learning

Python lists become inefficient.

NumPy provides:

- Efficient numerical arrays
- Fast computation

- Vectorized operations
- Memory optimization

NumPy stands for:

Numerical Python

```
# First, import NumPy

import numpy as np
```

3. Creating a NumPy Array

The basic syntax:

np.array(object)

We usually convert a list into a NumPy array.

```
# Creating a 1D NumPy array

arr = np.array([10, 20, 30, 40])

print("NumPy Array:", arr)
print("Type:", type(arr))
```

```
NumPy Array: [10 20 30 40]
Type: <class 'numpy.ndarray'>
```

Observation:

The type is:

<class 'numpy.ndarray'>

ndarray means n-dimensional array.

---

4. Difference Between List and NumPy Array

```
# Python list
list_data = [1, 2, 3, 4]

# NumPy array
array_data = np.array([1, 2, 3, 4])

print("List:", list_data)
```

```
print("Array:", array_data)
```

```
List: [1, 2, 3, 4]
Array: [1 2 3 4]
```

Now let us try adding 5 to each element.

```
# List operation (requires loop)

result_list = []
for i in list_data:
    result_list.append(i + 5)

print("Updated List:", result_list)

# NumPy operation (vectorized)

print("Updated Array:", array_data + 5)
```

```
Updated List: [6, 7, 8, 9]
Updated Array: [6 7 8 9]
```

Important Observation:

- NumPy performs element-wise operations automatically.
- This is called vectorization.

Vectorization:

Applying an operation to all elements without writing an explicit loop.

---

5. Creating Multi-Dimensional Arrays

NumPy supports multi-dimensional arrays.

```
# 2D Array (Matrix)

matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])

print("2D Array:")
print(matrix)
```

```
2D Array:
[[1 2 3]
```

```
[4 5 6]]
```

6. Checking Array Properties

Important properties:

- shape
- ndim
- size
- dtype

```
print("Shape:", matrix.shape)
print("Number of dimensions:", matrix.ndim)
print("Total elements:", matrix.size)
print("Data type:", matrix.dtype)
```

```
Shape: (2, 3)
Number of dimensions: 2
Total elements: 6
Data type: int64
```

Explanation:

shape → (rows, columns)

ndim → number of dimensions

size → total elements in array

dtype → data type of elements

---

7. Creating Special Arrays

```
# Array of zeros
zeros_array = np.zeros((2, 3))
print("Zeros Array:")
print(zeros_array)

# Array of ones
ones_array = np.ones((3, 2))
print("Ones Array:")
print(ones_array)

# Range array
range_array = np.arange(1,10000,2)
print("Range Array:", range_array)
```

```
Zeros Array:
[[0. 0. 0.]
 [0. 0. 0.]]
Ones Array:
[[1. 1.]
 [1. 1.]
 [1. 1.]]
Range Array: [   1    3    5 ... 9995 9997 9999]
```

8. Basic Mathematical Operations

NumPy supports element-wise mathematical operations.

```
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])

print("Addition:", arr1 + arr2)
print("Subtraction:", arr1 - arr2)
print("Multiplication:", arr1 * arr2)
print("Division:", arr1 / arr2)
```

```
Addition: [5 7 9]
Subtraction: [-3 -3 -3]
Multiplication: [ 4 10 18]
Division: [0.25 0.4  0.5 ]
```

9. In-Class Practice Problems

Students should attempt the following during the lecture.

Problem 1: Create a NumPy array containing numbers from 1 to 20.

Problem 2: Create a 3x3 matrix with values from 1 to 9.

Problem 3: Multiply every element of an array by 10.

Problem 4: Find the shape and dimension of a 4x5 zero matrix.

```
# Problem 1
arr_p1 = np.arange(1, 21)
print("Problem 1:", arr_p1)

# Problem 2
arr_p2 = np.arange(1, 10).reshape(3, 3)
print("Problem 2:")
print(arr_p2)

# Problem 3
arr_p3 = np.array([2, 4, 6])
print("Problem 3:", arr_p3 * 10)
```

```
# Problem 4
arr_p4 = np.zeros((4, 5))
print("Problem 4 Shape:", arr_p4.shape)
print("Problem 4 Dimension:", arr_p4.ndim)
```

```
Problem 1: [ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20]
Problem 2:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Problem 3: [20 40 60]
Problem 4 Shape: (4, 5)
Problem 4 Dimension: 2
```

Final Summary of Today's Learning

We understood:

- What is an array
- Why NumPy is required
- How to create NumPy arrays
- Difference between list and array
- Vectorized operations
- Multi-dimensional arrays
- Array properties
- Basic mathematical operations

This forms the foundation of:

- Data Science
- Machine Learning
- Scientific Computing
- Deep Learning

SESSION 2 — Creating Arrays in NumPy

NumPy provides multiple functions to create arrays efficiently.

Why do we need different creation functions?

Because in real-world computation we frequently need:

- Sequential data
- Fixed spacing data
- Logarithmic spacing

- Identity matrices
- Pre-filled arrays
- Random data for simulations
- Memory-efficient initialization

Each function exists to solve a specific computational requirement.

---

1. np.array()

Why It Exists

To convert Python sequences (list, tuple) into NumPy arrays.

This is the most basic way to create an array.

Syntax

np.array(object, dtype=None)

Parameters

- object ⟶ list, tuple, or sequence
- dtype ⟶ optional, specify data type

Advantage

- Converts existing data into optimized NumPy structure
- Allows control over data type

```
import numpy as np

arr = np.array([1, 2, 3, 4], dtype=float)

print(arr)
print(arr.dtype)
```

```
[1. 2. 3. 4.]
float64
```

2. np.arange()

Why It Exists

To create arrays with evenly spaced values within a given interval.

Equivalent to Python's range(), but returns a NumPy array.

Syntax

np.arange(start, stop, step, dtype=None)

Parameters

- start ⟶ starting value (inclusive)
- stop ⟶ ending value (exclusive)
- step ⟶ spacing between values
- dtype ⟶ optional data type

Advantage

- Fast creation of numeric sequences
- Memory efficient
- Used extensively in simulations

```
arr = np.arange(0, 10, 2)
print(arr)
```

```
[0 2 4 6 8]
```

3. np.linspace()

Why It Exists

To create evenly spaced numbers between two limits.

Used when exact number of samples is required.

Syntax

np.linspace(start, stop, num=50, endpoint=True, dtype=None)

Parameters

- start ⟶ starting value
- stop ⟶ ending value
- num ⟶ number of samples
- endpoint ⟶ include stop value or not
- dtype ⟶ optional data type

Advantage

- Guarantees fixed number of points
- Useful in plotting and numerical analysis

```
arr = np.linspace(0, 10, 5)
print(arr)
```

```
[ 0.   2.5  5.   7.5 10. ]
```

4. np.logspace()

Why It Exists

To generate numbers spaced evenly on a logarithmic scale.

Used in signal processing and scientific computation.

Syntax

np.logspace(start, stop, num=50, base=10.0)

Parameters

- start ⟶ exponent start
- stop ⟶ exponent stop
- num ⟶ number of samples
- base ⟶ logarithmic base

Advantage

- Essential for exponential growth modeling
- Used in frequency analysis

```
arr = np.logspace(1, 3, 4)
print(arr)
```

```
[  10.          46.41588834  215.443469   1000.        ]
```

5. np.zeros()

Why It Exists

To initialize arrays with zeros.

Common in machine learning for weight initialization.

Syntax

np.zeros(shape, dtype=float)

Parameters

- shape ⟶ tuple specifying dimensions
- dtype ⟶ optional

Advantage

- Fast initialization
- Cleaner than manual creation

```
arr = np.zeros((2, 3))
print(arr)
```

```
[[0. 0. 0.]
 [0. 0. 0.]]
```

```
Start coding or generate with AI.
```

6. np.ones()

Creates array filled with ones.

Syntax:

np.ones(shape, dtype=None)

Used in scaling and initialization tasks.

```
arr = np.ones((3, 2))
print(arr)
```

```
[[1. 1.]
 [1. 1.]
 [1. 1.]]
```

7. np.empty()

Why It Exists

To create an array without initializing entries.

Faster than zeros and ones because values are not assigned.

Syntax

np.empty(shape, dtype=float)

Advantage

- Faster for large arrays
- Used when values will be overwritten

```
arr = np.empty((3,3))
print(arr)
```

```
[[7.9730972e-316 0.0000000e+000 0.0000000e+000]
 [0.0000000e+000 0.0000000e+000 0.0000000e+000]
 [0.0000000e+000 0.0000000e+000 0.0000000e+000]]
```

8. np.eye() and np.identity()

Why They Exist

To create identity matrices.

Used in linear algebra and matrix operations.

Syntax

np.eye(N, M=None)

np.identity(n)

Parameters:

- N → number of rows
- M → columns (optional)

Advantage:

- Essential in matrix multiplication
- Used in solving linear equations

```
print(np.eye(2,4))
print(np.identity(4))
```

```
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]]
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

`# 9. Random Array Generation

Why random arrays exist?

- Simulations
- Machine learning
- Statistical experiments
- Testing algorithms

---

np.random.rand()

Uniform distribution [0,1)

Syntax: np.random.rand(size)

---

np.random.randn()

Normal distribution (mean=0, std=1)

Syntax: np.random.randn(size)

---

np.random.randint()

Random integers

Syntax: np.random.randint(low, high, shape)

---

np.random.choice()

Random sampling

Syntax: np.random.choice(a, size=None)

---

np.random.seed()

Ensures reproducibility

Syntax: np.random.seed(seed_value)

```
`
```

```
np.random.seed(42)

print("Uniform:", np.random.rand(2,2))
print("Normal:", np.random.randn(2,2))
print("Random Integers:", np.random.randint(1, 10, (2,3)))
print("Random Choice:", np.random.choice([10,20,30], size=9))
```

```
Uniform: [[0.37454012 0.95071431]
 [0.73199394 0.59865848]]
Normal: [[-0.23415337 -0.23413696]
 [ 1.57921282  0.76743473]]
Random Integers: [[6 5 2]
 [8 6 2]]
Random Choice: [10 10 20 20 10 10 10 30 30]
```

SESSION 3 — Array Attributes

These properties help understand memory and structure.

---

1. ndim

Number of dimensions.

2. shape

Tuple representing dimensions.

3. size

Total number of elements.

4. dtype

Data type of elements.

5. itemsize

Size of one element (in bytes).

6. nbytes

Total memory consumed.

```
arr = np.array([[1,2,3],[4,5,6]])

print("ndim:", arr.ndim)
print("shape:", arr.shape)
print("size:", arr.size)
```

```
print("dtype:", arr.dtype)
print("itemsize:", arr.itemsize)
print("nbytes:", arr.nbytes)
```

```
ndim: 2
shape: (2, 3)
size: 6
dtype: int64
itemsize: 8
nbytes: 48
```

Reshaping Methods

reshape()

Changes dimensions without changing data.

Syntax: arr.reshape(new_shape)

---

ravel()

Returns flattened array (view).

---

flatten()

Returns flattened copy.

---

resize()

Changes shape permanently.

```
arr = np.arange(1, 10)
print(arr)

reshaped = arr.reshape(3,3)
print("Reshaped:\n", reshaped)
print(arr)

print("Ravel:", reshaped.ravel())
print(arr)

print("Flatten:", reshaped.flatten())
print(arr)

arr.resize(3,3)
print("Resize:\n", arr)
print(arr)
```

```
[1 2 3 4 5 6 7 8 9]
Reshaped:
 [[1 2 3]
 [4 5 6]
 [7 8 9]]
[1 2 3 4 5 6 7 8 9]
Ravel: [1 2 3 4 5 6 7 8 9]
[1 2 3 4 5 6 7 8 9]
Flatten: [1 2 3 4 5 6 7 8 9]
[1 2 3 4 5 6 7 8 9]
Resize:
 [[1 2 3]
 [4 5 6]
 [7 8 9]]
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Transpose

arr.T

Shortcut for transpose.

np.transpose(arr)

Explicit transpose function.

Used to swap rows and columns.

```
matrix = np.array([[1,2,3],[4,5,6]])

print("Original:\n", matrix)
print("Transpose using T:\n", matrix.T)
print("Transpose using function:\n", np.transpose(matrix))
```

```
Original:
 [[1 2 3]
 [4 5 6]]
Transpose using T:
 [[1 4]
 [2 5]
 [3 6]]
Transpose using function:
 [[1 4]
 [2 5]
 [3 6]]
```

NumPy Practice Exercise Set

This exercise set is based on:

- Array creation methods
- Random arrays
- Array attributes
- Reshaping methods
- Transpose operations

Section A — Easy Level Questions

1. Create a NumPy array using np.array() with elements [5, 10, 15, 20].
2. Create an array from 0 to 18 with step size 3 using np.arange().
3. Create 6 evenly spaced numbers between 1 and 3 using np.linspace().
4. Create a 2×4 zero matrix.
5. Create a 3×3 matrix filled with ones.
6. Generate a 2×2 matrix of random numbers from uniform distribution.
7. Create an identity matrix of size 4.
8. Find the shape, size, and number of dimensions of the following array:

   np.array([[1,2,3],[4,5,6]])

Section B — Medium Level Questions

9. Create an array from 1 to 12 and reshape it into a 3×4 matrix.
10. Create a 4×4 matrix using np.arange() and transpose it.
11. Generate 5 logarithmically spaced numbers between 10^1 and 10^3.

12. Create a random integer matrix of shape 3×3 with values between 10 and 50.
13. Create a 2×5 matrix using np.random.randn() and compute:
    - Total memory consumed (nbytes)
    - Size of one element (itemsize)
14. Create an array of numbers from 1 to 9 and:
    - Flatten it using ravel()
    - Flatten it using flatten()
    - Explain the difference